



APIs de IA — Seguridad, Diseño de Apps y Pipeline Integrado

Ejercicio de Clase · Ejercicio 3 de 3 · Bloques 4.4 — 4.5 — 4.8 — Integración Pipeline

Nombre del alumno/a	
Fecha	
 Enlace GitHub del repositorio	

 **Instrucciones:** Ejercicio práctico. Diseña soluciones reales usando APIs de IA integrando lo que ya sabes de modularización y GitHub Actions. Razona tus decisiones técnicas en cada bloque.

BLOQUE 1 — Seguridad: buenas prácticas y errores comunes (4.8)

En clase vimos los errores más frecuentes y las buenas prácticas al trabajar con APIs de IA. Completa la tabla:

Buenas Prácticas 	Errores Comunes 
Nunca hardcodear la API key en el código fuente	Subir la API key directamente en el código a GitHub

1.2 — Detecta el error de seguridad

Lee el siguiente fragmento de código. Identifica TODOS los problemas de seguridad que encuentras y cómo los corregirías:

```
# archivo: app.py - subido a GitHub público
from openai import OpenAI

API_KEY = 'sk-proj-abc123xyz789...'
```

```

client = OpenAI(api_key=API_KEY)

def responder(pregunta_usuario):
    while True: # bucle sin límite
        response = client.chat.completions.create(
            model='gpt-4o', # modelo más caro
            messages=[{'role': 'user', 'content': pregunta_usuario}]
        )
        print(response.choices[0].message.content)

```

Problema detectado	¿Cómo lo corregirías?

1.3 — Variables de entorno y GitHub Secrets

Ordena del 1 al 6 los pasos correctos para gestionar una API key de forma segura en un proyecto con GitHub Actions:

Orden	Paso a seguir
	Añadir el archivo .env al .gitignore para que nunca se suba al repositorio.
	Crear el archivo .env local con OPENAI_API_KEY=sk-proj-tu-clave.
	En el código, leer la clave con os.getenv('OPENAI_API_KEY').
	En GitHub, ir a Settings → Secrets and variables → Actions y añadir el Secret.
	En el archivo .yml del workflow, pasar el Secret como variable de entorno al step.
	Verificar en el pipeline que la variable está disponible antes de ejecutar el script.

BLOQUE 2 — Diseña una aplicación real con APIs de IA (4.4 + 4.5)

 **Caso práctico:** Tu equipo quiere construir un sistema de soporte técnico automatizado para una empresa de software. El sistema debe: (1) clasificar tickets de soporte por categoría y urgencia, (2) generar una respuesta inicial automática, (3) detectar si el problema es conocido buscando en la base de conocimiento por significado. Tienes un presupuesto limitado y el sistema recibirá ~500 tickets/día.

Diseña la arquitectura de la aplicación completando la siguiente tabla:

Funcionalidad	API / Proveedor elegido + Modelo	Justificación técnica
Clasificación de tickets (categoría + urgencia)		
Generación de respuesta inicial automática		
Búsqueda semántica en base de conocimiento		
Análisis de capturas de pantalla adjuntas		

2.2 — Diseño modular de la aplicación

Usando lo que ya sabes de modularización, divide esta aplicación en módulos. Para cada módulo indica su responsabilidad y qué función de la API utiliza:

Nombre del módulo	Responsabilidad única (SRP)	Llamada a la API que realiza

2.3 — Optimización de costes

Con 500 tickets/día, el coste puede dispararse. Describe 4 estrategias concretas para optimizar el coste de esta aplicación:

Nº	Estrategia de optimización	¿Cómo se implementa en el código/pipeline?
1		
2		
3		

4		
---	--	--

BLOQUE 3 — Pipeline CI/CD integrado con APIs de IA (GitHub Actions)

Ya sabes diseñar workflows con GitHub Actions. Ahora integra una API de IA en el pipeline de la aplicación de soporte del Bloque 2.

3.1 — Diseña el workflow de GitHub Actions

Describe los steps del workflow .yml que automatizaría el despliegue y la integración de la API de IA. NO necesitas escribir el YAML exacto, solo describe cada step y su propósito:

Step	Nombre del paso	¿Qué hace? ¿Por qué es necesario?
1		
2		
3		
4		
5		
6		
7		

3.2 — Verdadero o Falso: Pipeline con APIs de IA

Marca V (Verdadero) o F (Falso) y justifica brevemente las que sean Falsas:

V / F	Afirmación	Justificación si es Falsa
	Las API keys deben almacenarse como GitHub Secrets, nunca en el código ni en el .yml directamente.	
	Un pipeline puede disparar miles de llamadas a la API si hay un bucle infinito o una mala configuración.	
	El verdadero reto del coste no es el precio por llamada, sino controlar el uso descontrolado.	
	Si el proveedor de IA sufre una caída, el pipeline con manejo de errores puede hacer reintento automático.	

	Gemini Flash y GPT-4o-mini son los modelos más recomendados para pipelines de alta frecuencia.	
	Una vez integrada la API en el pipeline, ya no hace falta monitorizar el consumo.	

BLOQUE 4 — Manejo de errores en llamadas a la API (4.8)

Las APIs de IA pueden fallar por múltiples razones. Completa la tabla indicando qué causa cada error y cómo lo manejarías en el código:

Tipo de error	¿Qué lo causa?	¿Cómo lo manejarías?
Rate limit (429)		
Timeout del servidor		
Error de autenticación (401)		
Prompt demasiado largo (400)		
Error del servidor del proveedor (5xx)		
Respuesta en formato inesperado		

BLOQUE 5 — Proyecto integrador final

 Situación real: Eres el desarrollador principal de una startup que quiere lanzar en 2 semanas un MVP (producto mínimo viable) de un asistente de IA para gestión de contratos legales. Debe: analizar contratos PDF, detectar cláusulas de riesgo, responder preguntas sobre el contrato, y enviar un resumen ejecutivo por email. La empresa ya usa GitHub y tiene conocimientos de Python y GitHub Actions.

Diseña tu estrategia técnica completa respondiendo estas preguntas clave:

Decisión técnica	Tu respuesta justificada
¿Qué proveedor y modelo elegirías para analizar contratos largos? ¿Por qué?	
¿Qué parámetros clave configurarías (temperature, max_tokens, system prompt)?	
¿Cómo estructurarías los módulos de la aplicación?	

¿Cómo gestionarías la API key en el repositorio y en el pipeline?	
¿Qué harías para controlar que el coste no se dispare?	
¿Qué errores anticipas y cómo los manejarías?	